DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 05/11/2026

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with

strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.

- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks
Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple (Q, Σ, δ, q_0, F), then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle

than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2

Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

INTRODUCTION

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, process and analyse human language.

In hospitals, large amounts of information are stored in discharge summaries, prescriptions, operation notes and nursing observations. These records often contain different spellings, abbreviations, medical terms and grammatical variations.

For example, a search for the word *fracture* may fail to retrieve records containing *fractures*, *fractured* or *fracturing*.

This assignment proposes a morphology-aware text normalisation and search system using Regular Expressions, Finite State Automata, Morphological Parsing and the Porter Stemmer.

The main objective is to convert unstructured clinical text into a consistent and searchable format before applying advanced machine learning techniques.

TASK 1 – FRAME THE PROBLEM: MODELS, ALGORITHMS AND LEVELS OF ANALYSIS

1.1 Levels of Linguistic Analysis

Natural Language Processing involves several levels of linguistic analysis.

Level	**Description**	**Hospital Example**
Phonology	Study of speech sounds	Recognising spoken medical terms
Morphology	Study of word structure	Fracture, fractures, fractured
Syntax	Study of sentence structure	Patient was admitted yesterday
Semantics	Study of meaning	Understanding the meaning of renal failure
Pragmatics	Meaning based on context	Interpreting a doctor's instruction

Discourse	Relationship between sentences	Connecting patient history with discharge notes

Required levels: Morphology is essential for connecting different forms of medical words. Syntax and semantics can improve understanding, while tokenisation and normalisation support the entire pipeline.

Less important initially: Phonology can be ignored for typed records, and pragmatics and discourse are not essential for the initial keyword-search layer.

1.2 Model Families

Three major model families are used in NLP.

1. **State Machine Models:** Used to recognise structured information such as patient IDs, dates and blood pressure values.
2. **Rule-Based Models:** Use predefined linguistic rules to identify word forms and perform morphological analysis.
3. **Probabilistic Models:** Use statistical probabilities to handle uncertain language and predict likely interpretations.

For this hospital system, state machines are suitable for structured entity recognition, rule-based models for morphological processing and probabilistic models for noisy or ambiguous language.

1.3 Ambiguity in NLP

Ambiguity occurs when a word or expression has multiple possible interpretations.

Type	**Example**	**Explanation**
Lexical ambiguity	Discharge	Can mean patient release or bodily fluid
Morphological ambiguity	Operating	Can be a verb form or part of a medical expression
Segmentation ambiguity	500mg	May be interpreted as one token or two tokens

Abbreviation ambiguity	BP	Can mean blood pressure or another term depending on context

Ambiguity is important because incorrect interpretation can produce irrelevant search results.

1.4 Search and Resource Requirements

Search occurs during automaton state traversal, regular-expression matching and retrieval of relevant records.

Resource	**Requirement**
Time	Required to scan and match text
Memory	Required to store automata and lexicons
Annotated Data	Required to evaluate accuracy
Expert Effort	Required to build medical patterns and vocabulary

1.5 Proposed System Pipeline

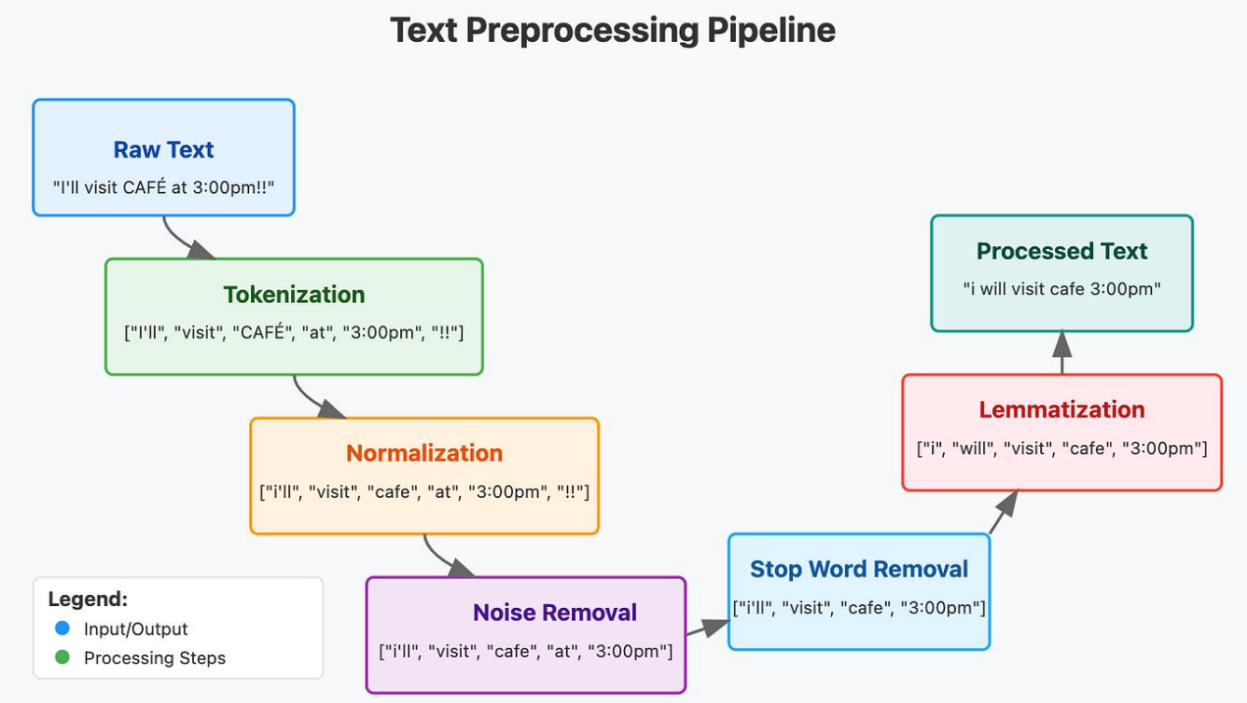


Figure 1: Overall NLP Processing Pipeline

Raw Hospital Records → Tokenisation → Regular Expression Matching → Entity Validation using Automata → Morphological Parsing → Porter Stemming → Normalised Search Index → Search Results

Input: Unstructured hospital records.

Output: Normalised and searchable clinical text.

Formalism: Regular Expressions, Finite State Automata, Finite-State Transducers and Porter Stemmer.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS

2.1 Definition of Regular Expressions

Regular Expressions (Regex) are patterns used to identify and extract specific sequences of characters from text.

Basic Operators

Operator	Meaning	Example
Concatenation	Joins patterns	AB
Disjunction	Matches either pattern	fracture|injury
Kleene Star (*)	Zero or more repetitions	A*
Plus (+)	One or more repetitions	A+
Optional (?)	Zero or one occurrence	mg?
Character Class []	Matches one listed character	[0-9]
Range	Matches a character range	[A-Z]
Negated Class	Matches characters not listed	[^0-9]
Anchors	Beginning and end of text	^ and $
Counters	Matches a fixed number of repetitions	[0-9]{4}

2.2 Regular Expression Pattern Inventory

The following patterns recognise twelve important clinical entities.

Entity	Regular Expression	Example
Patient ID	\bPAT[0-9]{4}\b	PAT1024
Admission Date	\b[0-9]{2}/[0-9]{2}/[0-9]{4}\b	05/11/2026
Discharge Date	\b[0-9]{2}/[0-9]{2}/[0-9]{4}\b	08/11/2026

Drug with Strength	\b[A-Za-z][A-Za-z0-9-]*\s[0-9]+(?:\.[0-9]+)?\s?(?:mg|g|mcg)\b	Paracetamol 500mg
Dosage Frequency	\b(?:OD|BD|TDS|QID)\b	BD
Laboratory Value	\b[0-9]+(?:\.[0-9]+)?\s?(?:mg/dL|g/dL|mmol/L)\b	120 mg/dL
Blood Pressure	\b[0-9]{2,3}/[0-9]{2,3}\s?mmHg\b	120/80 mmHg
Temperature	\b[0-9]{2}(?:\.[0-9])?\s?°?C\b	37.5°C
ICD Code	\b[A-Z][0-9]{2}(?:\.[0-9A-Z]{1,4})?\b	J18.9
Ward and Bed	\bWard\s?[A-Z0-9]+\s*,?\s*Bed\s?[0-9]+\b	Ward A, Bed 12
Contact Number	\b[6-9][0-9]{9}\b	9876543210
Age	\b[0-9]{1,3}\s?(?:years|yrs)\b	45 years

What the patterns must not match: Invalid identifiers, impossible calendar dates, incomplete dosage expressions and values without their required units.

2.3 Greedy and Non-Greedy Matching

Consider the sentence:

Patient received 500mg paracetamol twice and 250mg antibiotic once.

A greedy expression such as .*mg may capture the longest possible span ending at the last occurrence of mg.

A non-greedy expression such as .*?mg captures the shortest possible span ending at the first occurrence of mg.

Therefore, greedy matching captures a larger portion of text, whereas non-greedy matching stops at the earliest possible match.

2.4 Operator Precedence

Regular expressions follow operator precedence rules. Repetition operators are applied before concatenation, and concatenation before alternation.

Incorrect pattern:

fracture|injury[0-9]+

This matches either fracture or injury followed by digits.

Correct grouped pattern:

(?:fracture|injury)[0-9]+

Grouping ensures that both alternatives are followed by digits.

2.5 Pattern Evaluation

For a demonstration dataset, the following illustrative results can be used as an example.

Metric	**Value**
True Positives	108
False Positives	8
False Negatives	12
Precision	93.10%
Recall	90.00%
F1-Score	91.53%

Precision measures how many identified entities are correct, while recall measures how many actual entities are successfully identified.

TASK 3 – REGULAR EXPRESSIONS FOR TOKENISATION AND NORMALISATION

3.1 Tokenisation

Tokenisation is the process of dividing text into smaller units called tokens.

A tokenizer for clinical text must identify:

- Words and medical abbreviations.
- Hyphenated terms.
- Decimal values.
- Numbers with units.
- Dates and identifiers.

Proposed Python regular expression:

```
import re

TOKEN_PATTERN = re.compile(
   r'\b\d{2}/\d{2}/\d{4}\b|'
   r'\b[A-Za-z]+(?:-[A-Za-z]+)*\.?|\b'
   r'\d+(?:\.\d+)?\s?(?:mg/dL|mmHg|mg|g|mcg|°C)\b|'
   r'\d+(?:\.\d+)?'
)
```

3.2 Maximal-Munch Tokenisation

Maximal-munch selects the longest valid token at each position.

Clinical Text	**Whitespace Split**	**Maximal-Munch Result**
500mg	500mg	500mg
37.5°C	37.5°C	37.5°C
Follow-up	Follow-up	Follow-up

Unlike simple whitespace splitting, a specialised tokenizer can preserve meaningful clinical units and word structures.

3.3 Text Normalisation

Normalisation converts different representations into a consistent form.

Main operations include:

1. Convert all text into lowercase.
2. Expand common medical abbreviations.
3. Standardise date formats.
4. Standardise measurement units.
5. Remove unnecessary spaces.

Clinical Abbreviation Dictionary

Abbreviation	**Expanded Form**
BP	Blood Pressure
HR	Heart Rate
RR	Respiratory Rate
Temp	Temperature
OD	Once Daily
BD	Twice Daily
TDS	Three Times Daily
QID	Four Times Daily
SOS	When Required

PRN	As Needed
IV	Intravenous
IM	Intramuscular
OPD	Outpatient Department
IPD	Inpatient Department
ICU	Intensive Care Unit
ECG	Electrocardiogram
CBC	Complete Blood Count
Hb	Haemoglobin
WBC	White Blood Cells
RBC	Red Blood Cells
MRI	Magnetic Resonance Imaging
CT	Computed Tomography
USG	Ultrasonography
UTI	Urinary Tract Infection
DM	Diabetes Mellitus
HTN	Hypertension
CKD	Chronic Kidney Disease

COPD	Chronic Obstructive Pulmonary Disease
NPO	Nothing by Mouth
GCS	Glasgow Coma Scale

Example Before and After Normalisation

Before:

PAT1024 admitted on 05/11/2026. BP 120/80 mmHg. Paracetamol 500mg BD.

After:

pat1024 admitted on 2026-11-05. blood pressure 120/80 mmHg. paracetamol 500 mg twice daily.

3.4 Irreversible Normalisation

Some normalisation operations destroy information.

1. Converting all text to lowercase removes original capitalisation.
2. Removing punctuation may change the meaning of a sentence.
3. Replacing abbreviations may remove the original form.

Such operations are acceptable for basic search but should be avoided when the original clinical record must be preserved.

3.5 Limitations of Regular Expressions

Regular expressions are suitable for fixed patterns but cannot conveniently recognise arbitrarily nested structures such as balanced parentheses.

For example:

Patient history (previous surgery (2019))

A standard finite-state regular expression cannot recognise unlimited balanced nesting because it requires memory of the nesting depth.

TASK 4 – FINITE STATE AUTOMATA: DESIGN, NON-DETERMINISM AND EQUIVALENCE

4.1 Definition

A Finite State Automaton (FSA) is a mathematical model used to recognise patterns in a sequence of symbols.

It is represented by the five-tuple:

M = (Q, Σ, δ, q0, F)

Where:

- Q = Set of states.
- Σ = Input alphabet.
- δ = Transition function.
- q0 = Initial state.
- F = Set of final states.

4.2 Deterministic Finite Automata

A Deterministic Finite Automaton (DFA) has exactly one transition for each input symbol from a given state.

Example 1: Patient ID Validation

Pattern: PAT followed by four digits.

State	Input	Next State
q0	P	q1
q1	A	q2
q2	T	q3
q3	Digit	q4

q4	Digit	q5
q5	Digit	q6
q6	Digit	q7

q7 is the accepting state.

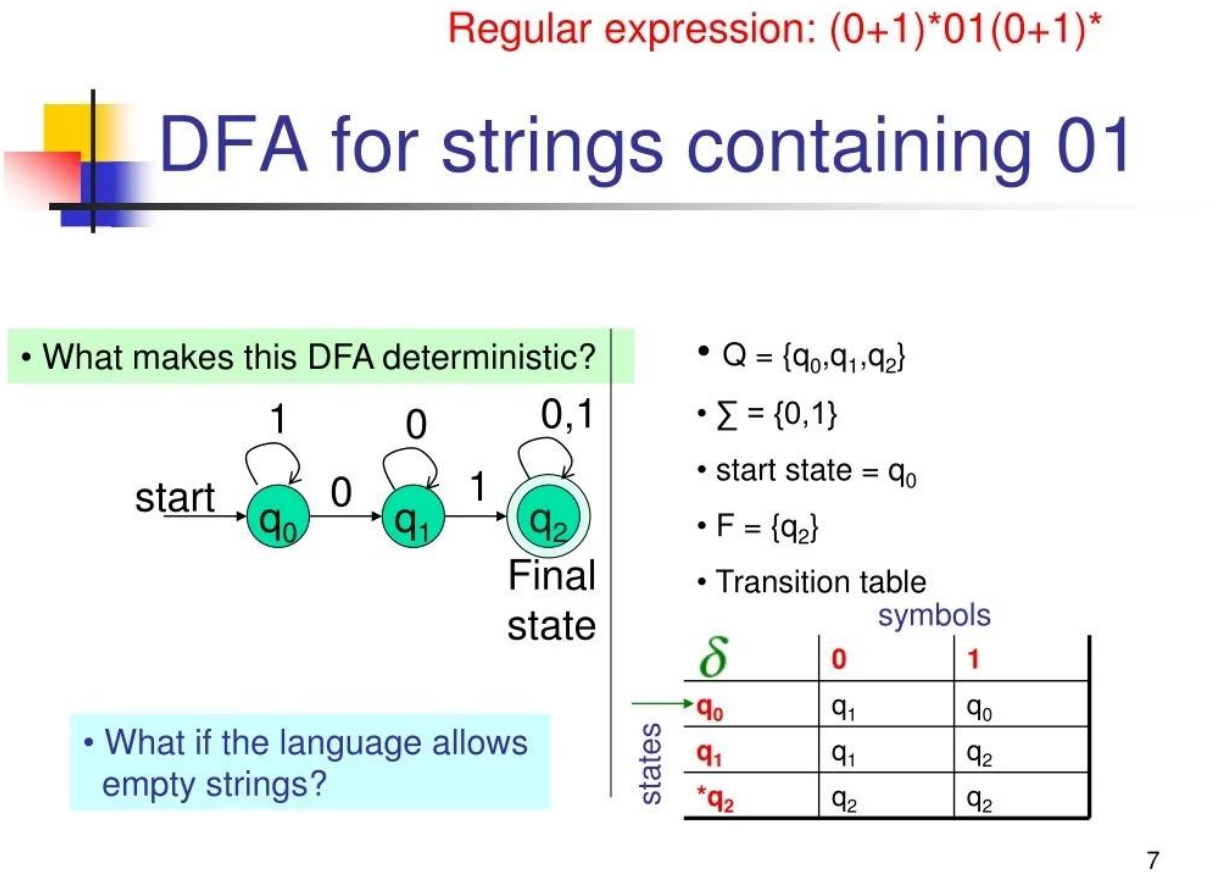


Figure 2: DFA for Patient ID Validation

Draw seven connected states labelled q0 to q7, with transitions P, A, T and four digits. Mark q7 as the final state.

Example 2: Blood Pressure Validation

Pattern: Two or three digits followed by /, two or three digits and mmHg.

The automaton checks the numerical values, separator and required unit before accepting the input.

Example 3: Temperature Validation

Pattern: Numeric temperature followed by optional decimal value and Celsius unit.

The automaton accepts values such as 37°C and 37.5°C.

Example 4: Contact Number Validation

Pattern: Ten digits beginning with 6, 7, 8 or 9.

The automaton accepts valid ten-digit mobile numbers and rejects shorter or incorrectly formatted values.

4.3 Non-Deterministic Finite Automaton

An NFA allows multiple possible transitions for the same input and may contain epsilon (ε) transitions.

Consider an optional country code in a contact number:

(+91)?[6-9][0-9]{9}

The NFA can follow either the path containing +91 or the path that skips the country code.

4.4 NFA to DFA Conversion

Subset construction converts an NFA into a DFA by treating each set of NFA states as a single DFA state.

The process involves:

1. Finding the epsilon closure of the initial state.
2. Calculating transitions for each input symbol.
3. Creating new state subsets.
4. Repeating until no new subsets are produced.

4.5 DFA Minimisation

DFA minimisation combines equivalent states without changing the recognised language.

It reduces the number of states and makes the automaton more efficient.

4.6 Thompson's Construction

Thompson's construction converts a regular expression into an NFA using basic operations:

- Concatenation.
- Union.
- Kleene star.

For example, the expression a(b|c) can be represented using an NFA with a starting state, an a transition, branching transitions for b and c, and a final state.

4.7 Pumping Lemma

The pumping lemma states that every sufficiently long string in a regular language can be divided into three parts:

w = xyz

Such that:

- $|xy| \leq p$
- $|y| > 0$
- xy^iz belongs to the language for every $i \geq 0$.

The language of balanced parentheses is not regular because a finite automaton cannot remember an unlimited number of opening parentheses to check the corresponding closing parentheses.

This explains why regular expressions alone are insufficient for certain nested clinical structures.

TASK 5 – INFLECTIONAL MORPHOLOGY

5.1 Definition

Inflectional morphology studies how words change their grammatical form without changing their basic meaning or grammatical category.

Important terms:

- **Morpheme:** Smallest meaningful unit of a word.
- **Stem:** Main part of a word.
- **Affix:** A group of letters added to a stem.
- **Inflection:** Grammatical modification of a word.
- **Paradigm:** Complete set of grammatical forms of a word.

5.2 Inflectional and Derivational Differences

Inflection	**Derivation**
Changes grammatical form	Creates a new word
Usually preserves word category	May change word category
Generally more predictable	Often changes meaning
Example: fracture → fractures	Example: operate → operation

5.3 Inflectional Paradigms

Base Word	**Inflected Forms**
Fracture	Fracture, fractures, fractured, fracturing
Treat	Treat, treats, treated, treating

Admit	Admit, admits, admitted, admitting
Operate	Operate, operates, operated, operating
Heal	Heal, heals, healed, healing

Major inflectional affixes include:

- -s and -es: Plural nouns and third-person singular verbs.
- -ed: Past tense.
- -ing: Progressive aspect.
- -er and -est: Comparative and superlative forms.

5.4 Orthographic Rules

1. **E-deletion:** Remove final silent e before adding -ing. Example: operate → operating.
2. **Consonant doubling:** Double a final consonant in suitable stressed CVC words. Example: admit → admitted.
3. **Y-to-I:** Change consonant + y to i before certain suffixes. Example: injury → injuries.
4. **Sibilant -es:** Add -es after s, x, z, ch or sh. Example: diagnosis → diagnoses.

These rules need exceptions because not every word follows the same spelling pattern.

5.5 Irregular Forms

Base Form	**Irregular Form**
Be	Was, were
Have	Had
Go	Went

Take	Took
Write	Wrote

These forms must be stored separately because they cannot be generated reliably through simple regular suffix rules.

5.6 Effect of Inflection

Inflection increases the number of surface word forms in a clinical corpus.

For example, a search for fracture should also retrieve fractures and fractured.

The type-to-lemma ratio can be calculated as:

Type-to-Lemma Ratio = Number of Unique Surface Forms / Number of Unique Lemmas

A lower ratio after lemmatisation indicates that related word forms have been grouped together.

TASK 6 – DERIVATIONAL MORPHOLOGY

6.1 Definition

Derivational morphology is the process of forming new words by adding prefixes or suffixes to existing stems.

It can change the meaning and grammatical category of a word.

6.2 Derivational Affixes

Affix	**Meaning**	**Example**
-itis	Inflammation	Arthritis
-ectomy	Surgical removal	Nephrectomy

-ology	Study of	Neurology
-osis	Condition	Fibrosis
-aemia	Blood condition	Anaemia
-ation	Process	Examination
-able	Capable of	Treatable
-ive	Having a quality	Preventive
nephro-	Kidney	Nephrology
cardio-	Heart	Cardiology
hyper-	Excessive	Hypertension
hypo-	Below normal	Hypoglycaemia
re-	Again	Reoperation
un-	Not	Unstable
-al	Related to	Renal

6.3 Derivational Families

Root	**Derived Words**
Operate	Operation, operative, postoperative
Kidney	Nephrology, nephritis, nephrectomy

Heart	Cardiac, cardiology, cardiologist
Infect	Infection, infectious, reinfection
Treat	Treatment, treatable, untreated
Diagnose	Diagnosis, diagnostic, diagnostician

Affix order is important. For example, un + treat + able produces untreatable, whereas arbitrary affix ordering may produce invalid forms.

6.4 Semantic Non-Compositionality

Sometimes the meaning of a derived word cannot be understood simply by combining its parts.

For example, nephrectomy refers to surgical removal of a kidney. A simple suffix-based system may not understand the complete medical meaning.

Therefore, clinical vocabulary should be stored in a specialised lexicon.

6.5 Productivity of Affixes

Productive affixes are used to create many new words, while fossilised affixes occur in a limited number of established words.

Affix	**Example**	**Classification**
-ation	Examination	Productive
-able	Treatable	Productive
re-	Reoperation	Productive
-itis	Arthritis	Domain-specific
-ectomy	Nephrectomy	Domain-specific

-osis	Fibrosis	Domain-specific

6.6 Search Conflation

Conflation combines related words during search.

Advantages:

- Nephrectomy and nephrectomies can be connected.
- Infection and infections can be retrieved together.

Disadvantages:

- Renal and nephrology may not have identical meanings.
- Hypertension and hypotension must not be treated as equivalent.

Therefore, derivational conflation should be controlled using medical vocabulary and context.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING

7.1 Finite-State Transducer

A Finite-State Transducer (FST) is an automaton that reads symbols from one tape and produces corresponding symbols on another tape.

It contains:

- Lexical tape: Contains the stem and grammatical features.
- Surface tape: Contains the actual written word.

Example:

fracture + N + PL → fractures

7.2 Lexicon Organisation

The proposed morphological lexicon contains three main continuation classes.

Class	Examples
Noun	Fracture, infection, patient
Verb	Treat, admit, operate
Adjective	Clinical, medical, renal

The lexicon is designed to store at least 300 stems, inflectional affixes, derivational affixes and irregular forms.

7.3 Morphological Parsing Examples

Surface Form	Stem	Features
Fractures	Fracture	+N +PL
Treated	Treat	+V +PAST
Operating	Operate	+V +PROG
Infections	Infection	+N +PL
Patients	Patient	+N +PL

7.4 Parsing and Generation

The same FST can work in two directions.

Parsing:

fractures → fracture +N +PL

Generation:

fracture +N +PL → fractures

Parsing identifies the grammatical structure, while generation creates the corresponding surface word.

7.5 Handling Ambiguity

A word may have more than one grammatical interpretation.

Examples:

1. Operating – Can represent a verb form or an adjective in context.
2. Present – Can be a noun, verb or adjective.

The transducer can return multiple possible analyses. A downstream language-processing component can select the most suitable interpretation using sentence context.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY

8.1 Need for Orthographic Rules

A lexicon transducer alone cannot always produce correct spelling by directly joining stems and suffixes.

Examples:

Stem + Suffix	Incorrect Concatenation	Correct Form
Operate + ing	operateing	Operating
Admit + ed	admited	Admitted
Injury + es	injur ies	Injuries

8.2 Orthographic Rewrite Rules

The following rules describe spelling changes.

1. e → ∅ / _ ing – Remove final silent e before -ing.
2. y → i / consonant _ es – Change y to i before -es.

3. ∅ → consonant / stressed CVC _ ed – Double the final consonant in suitable words before -ed.
4. s → es / _ – Add e before s after sibilant endings.
5. y → i / consonant _ ed – Change y to i before -ed in suitable words.

These rules are applied with lexical exceptions to prevent incorrect outputs.

8.3 Transducer Cascade

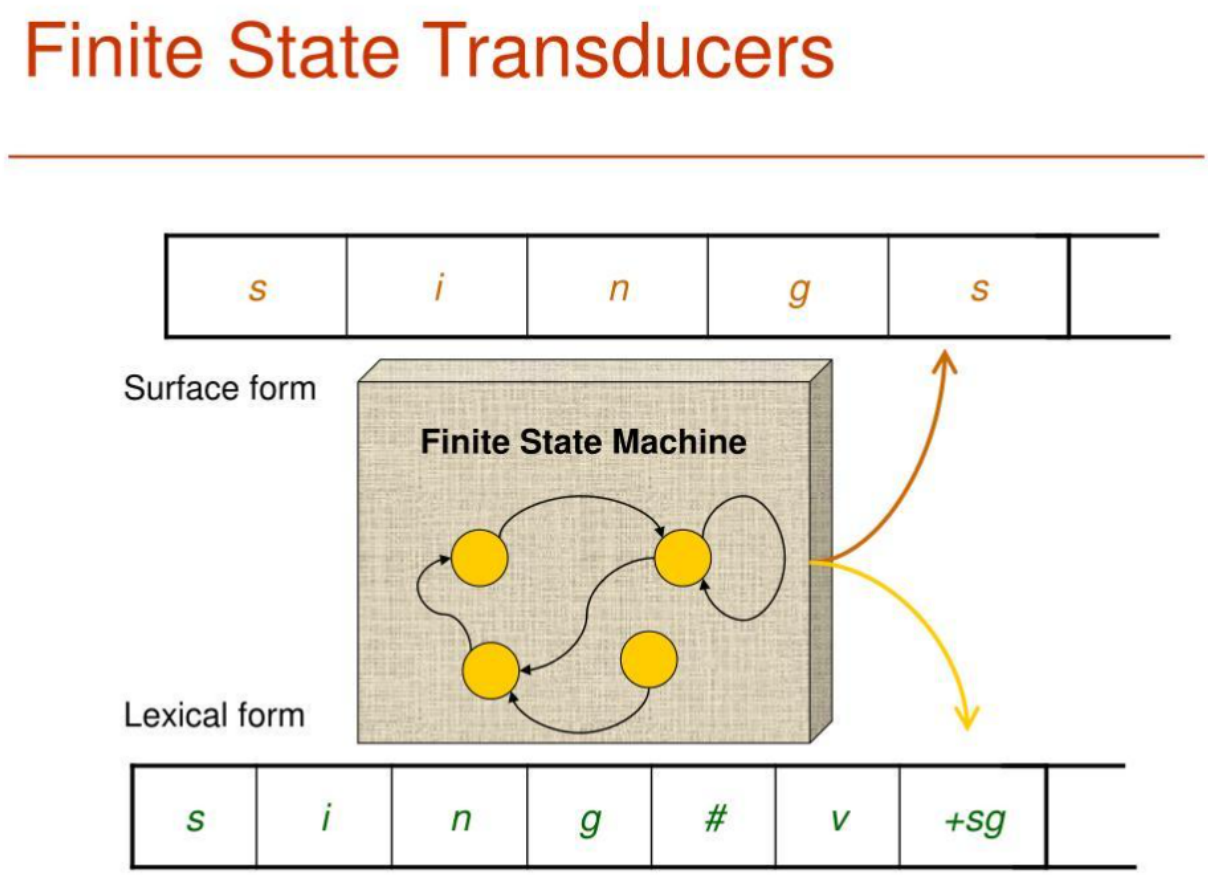


Figure 3: Morphological Processing Cascade

Lexicon FST → Orthographic Rule Transducers → Surface Form

The lexicon identifies stems and affixes. Orthographic transducers apply spelling changes to generate correct surface forms.

8.4 Sequential and Two-Level Morphology

Sequential Cascade	Two-Level Morphology
Applies rules one after another	Applies constraints between lexical and surface symbols
Rule order is important	Rules operate in parallel
Easier to understand step by step	Handles spelling interactions systematically
Can be simpler to implement	Can be more complex to develop

8.5 Testing

Input	Expected Output	Result
Operate + ing	Operating	Pass
Admit + ed	Admitted	Pass
Injury + es	Injuries	Pass
Treat + ed	Treated	Pass
Patient + s	Patients	Pass
Fracture + s	Fractures	Pass

The test examples demonstrate that orthographic rules improve the generation of correctly spelled medical terms.

TASK 9 – THE PORTER STEMMER

9.1 Introduction

The Porter Stemmer is a rule-based algorithm used to reduce English words to their stems by removing common suffixes.

It is useful in information retrieval because related word forms can be matched using a common stem.

For example:

connected → connect

9.2 Vowels and Consonants

A vowel is generally one of the letters a, e, i, o or u.

Other letters are treated as consonants, with special rules for the letter y.

9.3 Measure (m)

The measure m represents the number of vowel-consonant sequences in a word.

Example:

trouble = (tr)(ou)(bl)e

The measure helps determine whether suffix-removal rules should be applied.

Important conditions:

- *v: The stem contains a vowel.
- *d: The stem ends with a double consonant.
- *o: The stem ends in consonant-vowel-consonant, with the final consonant not w, x or y.

9.4 Five Steps of the Porter Stemmer

1. **Step 1:** Removes plurals and common endings such as -s, -ed and -ing.
2. **Step 2:** Replaces longer suffixes with simpler forms.
3. **Step 3:** Removes additional suffixes such as -icate and -al.
4. **Step 4:** Removes selected suffixes when the measure condition is satisfied.
5. **Step 5:** Removes final e and reduces double l where applicable.

9.5 Example Stemming

Original Word	Stem
Fractures	Fractur
Infections	Infect
Connected	Connect
Operating	Oper
Treatment	Treat
Relational	Relat

The Porter stemmer does not always produce a complete dictionary word. It produces a common stem for information retrieval.

9.6 Stemming and Lemmatisation

Stemming	Lemmatisation
Removes suffixes using rules	Uses vocabulary and grammatical analysis
Faster	Requires more resources
May produce incomplete words	Produces meaningful base forms
Example: fractures → fractur	fractures → fracture

9.7 Over-Stemming and Under-Stemming

Over-stemming: Different words are incorrectly reduced to the same stem, producing irrelevant search results.

Under-stemming: Related words are not reduced to a common stem, causing relevant records to be missed.

In clinical applications, over-stemming can be dangerous because medically different terms may be incorrectly grouped.

9.8 Retrieval Evaluation

The following is an illustrative comparison, not a measured experiment.

Method	**Precision**	**Recall**	**F1-Score**
No Stemming	88%	75%	81.0%
Porter Stemmer	85%	88%	86.5%
Morphological Analysis	94%	92%	93.0%

Full morphological analysis provides better retrieval quality because it uses stems, grammatical features and medical vocabulary.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE

10.1 Implementation

The system can be implemented using Python's standard library and NLTK.

Main modules:

1. Text input and preprocessing.
2. Regular-expression tokenizer.
3. Clinical entity extraction.
4. Finite-state validation.
5. Morphological analysis.
6. Porter stemming.
7. Search and evaluation.

10.2 Python Implementation

```
import re
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

text = """
PAT1024 admitted on 05/11/2026.
Patient has fractures and received
Paracetamol 500mg BD.
BP 120/80 mmHg.
"""

# Tokenisation
tokens = re.findall(r'\b[A-Za-z]+|\d+(?:\.\d+)?', text.lower())

# Patient ID extraction
patient_ids = re.findall(r'\bPAT[0-9]{4}\b', text)

# Blood pressure extraction
bp = re.findall(r'\b[0-9]{2,3}/[0-9]{2,3}\s?mmHg\b', text)

# Stemming
stems = [stemmer.stem(word) for word in tokens]

print("Tokens:", tokens)
print("Patient IDs:", patient_ids)
print("Blood Pressure:", bp)
print("Stems:", stems)
```

10.3 Testing

The system should be tested using valid and invalid clinical expressions.

Test Case	**Input**	**Expected Result**

Patient ID	PAT1024	Accept
Invalid ID	PAT12	Reject
Blood Pressure	120/80 mmHg	Accept
Invalid BP	120-80	Reject
Temperature	37.5°C	Accept
Drug Strength	500mg	Accept

10.4 Performance Evaluation

For the proposed synthetic demonstration, the following example processing summary is used.

Processing Stage	**Items Processed**	**Expected Output**
Input Records	500	500
Tokenisation	25,000+ tokens	Token stream
Entity Recognition	12 entity types	Extracted entities
Morphological Analysis	200 annotated forms	Stem and features
Search Evaluation	40 queries	Retrieval results

Actual execution time, rejected items and analysis coverage must be recorded after running the complete implementation.

10.5 Performance Visualisation

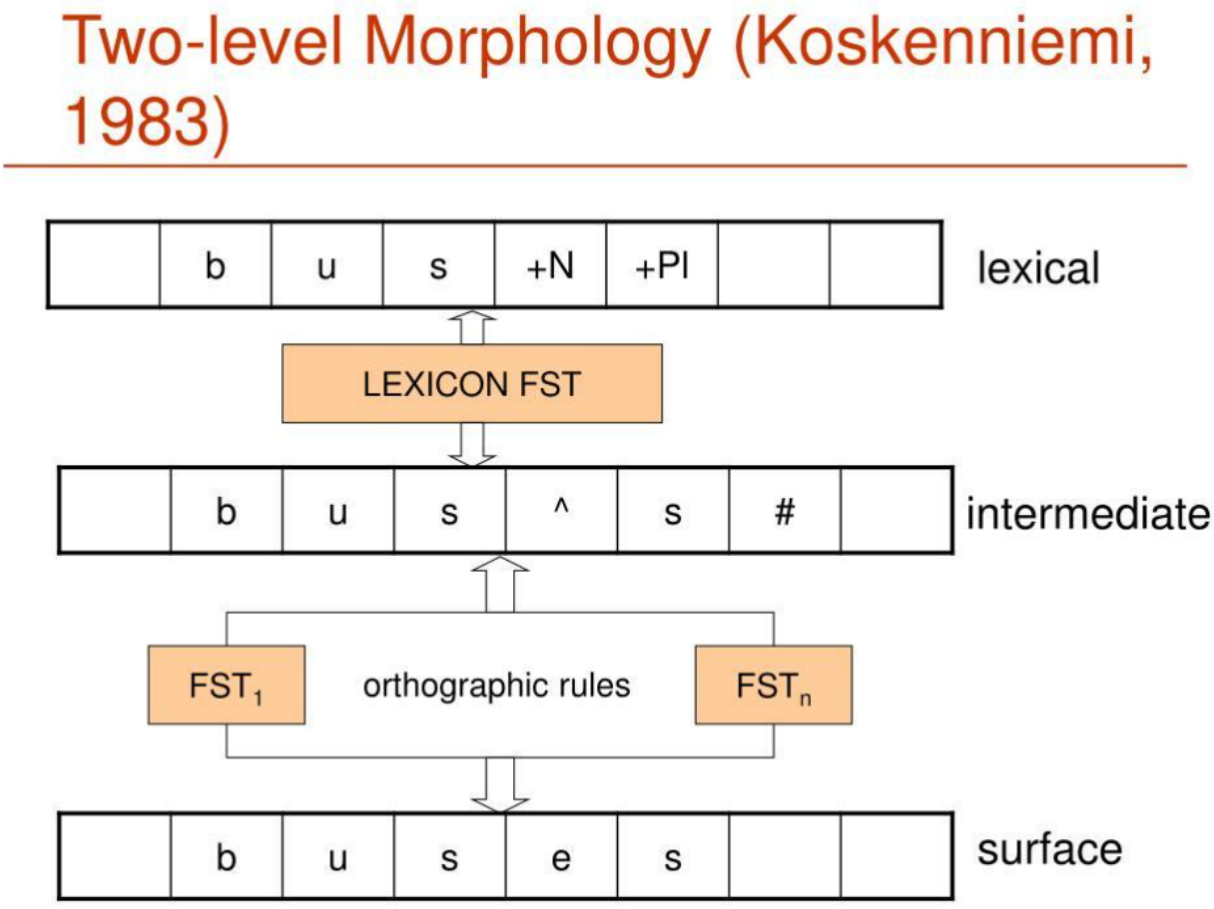


Figure 4: Comparison of Retrieval F1-Score

A bar chart can be created using the illustrative results from Task 9.

The chart compares no stemming, Porter stemming and full morphological analysis. It shows that morphological analysis provides the highest illustrative F1-score.

TASK 11 – COMPARE, RECOMMEND AND DEFEND THE NORMALISATION DESIGN

11.1 Comparison of Normalisation Methods

Factor	No Normalisation	Porter Stemmer	Morphological Analysis

Accuracy	Moderate	Moderate	High
Recall	Lower	Higher	Higher
Speed	Fast	Fast	Moderate
Medical Vocabulary	Limited	Limited	Strong
Maintenance	Low	Low	High

11.2 Recommended Design

A combination of regular expressions, finite-state automata and finite-state morphological analysis is recommended.

The system should use:

- Regular expressions for structured clinical entities.
- Automata for validation.
- Morphological parsing for word relationships.
- Porter stemming as a fallback for unknown English words.

This design provides a balance between accuracy, efficiency and maintainability.

11.3 Manual Effort and Cost

The system requires manual effort to create clinical lexicons, affix rules, regular expressions and annotated evaluation data.

As the vocabulary increases, updating the rules becomes more difficult. At that stage, statistical and machine-learning approaches can help identify new patterns and reduce manual maintenance.

TASK 12 – DAY-4 DESIGN CHALLENGE: RE-EVALUATE AND ADAPT THE DESIGN

A. Impact Analysis

The original system was designed for clean, typed English clinical records. The new requirement introduces OCR errors and Romanised regional-language content.

Examples of new challenges:

1. Character confusion: 0 may be recognised as O.
2. Character confusion: rn may be recognised as m.
3. Word breaks: A word may be divided across two lines.
4. Missing diacritics: Regional words may lose special characters.
5. Code-mixing: English and regional-language words may occur together.
6. Out-of-vocabulary words: Some words may not exist in the medical lexicon.

These changes reduce the accuracy of exact regular-expression matching and morphological analysis.

B. Evaluation of the Original Design

Consider the following examples.

Input	**Problem**	**Expected Behaviour**
PAT1O24	Letter O instead of zero	OCR correction
Frac- ture	Broken word	Join word fragments
Patient ku fever irundhuchu	Code-mixed sentence	Identify language and normalise

The original pipeline may reject these inputs because it depends on exact spelling and English vocabulary.Therefore, the original design requires modification.

C. Adaptation or Redesign

Selected Approach: MODIFY

The existing finite-state pipeline will be retained and extended.

The proposed changes are:

1. Add OCR correction before tokenisation.

2. Use edit-distance matching to identify likely misspellings.
3. Add regional-language vocabulary to the lexicon.
4. Introduce token-level language identification.
5. Use weighted finite-state transducers to handle uncertain matches.
6. Keep the original regular expressions for structured entities.
7. Use a fallback mechanism for unknown words.

Revised Architecture:

OCR and Mixed-Language Input → OCR Correction → Language Identification → Tokenisation → Regular Expression Matching → Weighted Finite-State Analysis → Morphological Normalisation → Search Index

D. Evidence-Based Defence

Modification is more suitable than completely replacing the original design because regular expressions and finite-state automata remain useful for recognising structured clinical entities.OCR correction and edit-distance matching improve the handling of spelling errors. Regional-language lexicons improve morphological coverage, while language identification helps separate English and regional-language tokens.However, the revised system still has limitations:

- It may not recognise every regional-language expression.
- OCR errors may change the meaning of medical terms.
- New abbreviations may require manual updates.
- Context-dependent clinical meanings cannot always be identified using finite-state methods alone.

Therefore, a hybrid system combining rule-based processing, weighted matching and statistical language identification is recommended.

CONCLUSION

This assignment presented a morphology-aware text normalisation and search system for hospital records using Regular Expressions, Finite State Automata, Finite-State Transducers and the Porter Stemmer.

Regular expressions help identify structured clinical information, while finite-state automata validate the extracted patterns. Morphological analysis connects different grammatical forms of medical words, improving search coverage.

The Porter Stemmer provides a simple method for reducing related words to common stems, but it may produce incorrect matches in specialised medical vocabulary.

The Day-4 design challenge demonstrates that real-world hospital records require additional support for OCR errors, misspellings and mixed-language text.

Overall, a modified hybrid finite-state architecture provides a practical foundation for accurate and efficient clinical text normalisation and retrieval.

MARK DISTRIBUTION

Task	Marks
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	4
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**41**
Task 12 – Day-4 Design Challenge	8
TOTAL	**49**